

Air Quality Forecasting

3 Model Results Analysis

Task 1: Air Quality Forecasting — Model Results & Analysis

****Target:**** one-hour-ahead `CO(GT)`

****Model:**** Random Forest Regressor

****Split:**** chronological 80% train / 20% test

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
DATA_PATH="/mnt/data/AirQualityUCI_submission.csv"
df=pd.read_csv(DATA_PATH, sep=None, engine="python")
df["Datetime"]=pd.to_datetime(df["Date"].astype(str)+" "+df["Time"].astype(str))
df=df.sort_values("Datetime").reset_index(drop=True)
num=df.select_dtypes(include=np.number).columns
df[num]=df[num].replace(-200, np.nan).ffill()

df["hour"]=df["Datetime"].dt.hour
df["day_of_week"]=df["Datetime"].dt.dayofweek
df["month"]=df["Datetime"].dt.month
df["is_weekend"]=(df["day_of_week"]>=5).astype(int)
df["target_next_CO"]=df["CO(GT)"].shift(-1)

lag_cols=["CO(GT)", "NOx(GT)", "NO2(GT)", "C6H6(GT)", "T", "RH", "AH"]
for c in lag_cols:
    df[f"{c}_lag1"]=df[c].shift(1)
    df[f"{c}_lag3"]=df[c].shift(3)
    df[f"{c}_lag24"]=df[c].shift(24)
for c in ["CO(GT)", "NOx(GT)", "NO2(GT)", "C6H6(GT)"]:
    df[f"{c}_roll3"]=df[c].shift(1).rolling(3).mean()
    df[f"{c}_roll24"]=df[c].shift(1).rolling(24).mean()

feature_cols=[c for c in df.columns if c not in ["Date", "Time", "Datetime", "target_next_CO"]]
model_df=df.dropna(subset=feature_cols+["target_next_CO"]).copy()
split_idx=int(len(model_df)*0.80)
train=model_df.iloc[:split_idx]; test=model_df.iloc[split_idx:]
X_train, y_train=train[feature_cols], train["target_next_CO"]
X_test, y_test=test[feature_cols], test["target_next_CO"]

model=RandomForestRegressor(n_estimators=200, max_depth=18, min_samples_leaf=2, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)
pred=model.predict(X_test)

mae=mean_absolute_error(y_test, pred); mse=mean_squared_error(y_test, pred)
rmse=np.sqrt(mse); r2=r2_score(y_test, pred)
results=pd.DataFrame({"Metric": ["MAE", "MSE", "RMSE", "R²"], "Value": [mae, mse, rmse, r2]})
display(results.round(4))
print("Training rows:", len(train), "Testing rows:", len(test))
print("Training:", train["Datetime"].min(), "to", train["Datetime"].max())
print("Testing:", test["Datetime"].min(), "to", test["Datetime"].max())
```

Actual vs predicted and error analysis

```
pred_df=test[["Datetime", "target_next_CO"]].copy()
pred_df["Prediction"]=pred
pred_df["Error"]=pred_df["target_next_CO"]-pred_df["Prediction"]
pred_df["Absolute_Error"]=pred_df["Error"].abs()
display(pred_df.head(15).round(3))

fig, ax=plt.subplots(figsize=(13, 5))
ax.plot(pred_df["Datetime"], pred_df["target_next_CO"], label="Actual", linewidth=0.9)
ax.plot(pred_df["Datetime"], pred_df["Prediction"], label="Predicted", linewidth=0.9)
ax.set_title("Actual vs Predicted Next-Hour CO"); ax.set_xlabel("Time"); ax.set_ylabel("CO(GT)")
ax.legend(); ax.grid(alpha=0.25); plt.tight_layout(); plt.show()

fig, ax=plt.subplots(figsize=(7, 6))
ax.scatter(pred_df["target_next_CO"], pred_df["Prediction"], alpha=0.35)
lims=[min(pred_df["target_next_CO"].min(), pred_df["Prediction"].min()), max(pred_df["target_next_CO"].max(), pred_df["Prediction"].max())]
ax.plot(lims, lims, linestyle="--")
ax.set_title("Actual vs Predicted CO"); ax.set_xlabel("Actual"); ax.set_ylabel("Predicted")
plt.tight_layout(); plt.show()

fig, ax=plt.subplots(figsize=(8, 5))
ax.hist(pred_df["Error"], bins=40); ax.axvline(0, linestyle="--")
ax.set_title("Prediction Error Distribution"); ax.set_xlabel("Actual - Predicted"); ax.set_ylabel("Frequency")
plt.tight_layout(); plt.show()

pred_df["Actual_Quartile"]=pd.qcut(pred_df["target_next_CO"], q=4, duplicates="drop")
display(pred_df.groupby("Actual_Quartile", observed=True)["Absolute_Error"].agg(["count", "mean", "median", "max"]).round(4))
```

Feature importance

```
importance=pd.Series(model.feature_importances_,index=feature_cols).sort_values(ascending=False)
display(importance.head(20).to_frame("Importance").round(4))
fig,ax=plt.subplots(figsize=(9,7))
importance.head(15).sort_values().plot(kind="barh",ax=ax)
ax.set_title("Top 15 Random Forest Feature Importances"); ax.set_xlabel("Importance")
plt.tight_layout(); plt.show()
```

Data leakage controls

- Chronological split: the final 20% is held out as future test data.
- The target is exactly one hour ahead.
- Lag variables use earlier observations only.
- Rolling variables are shifted before the rolling calculation.
- Missing values are forward-filled only.
- No random train/test shuffling is used.

5–7 Key findings

1. **Temporal dependence is important:** recent CO measurements and lag/rolling features provide useful information for one-hour-ahead forecasting.
2. **CO varies substantially over time:** the series contains pronounced short-term changes rather than a constant level.
3. **Time-of-day contributes information:** average CO differs across hours, supporting hour-based features.
4. **Pollutants are related:** other pollutant measurements provide additional predictive information.
5. **The model captures nonlinear relationships:** Random Forest can represent interactions among pollutant history, meteorological and calendar variables.
6. **Extreme observations are harder to predict:** larger absolute errors can occur for unusual/high concentration periods.
7. **The evaluation is forecasting-oriented:** chronological splitting and causal features make the test assessment more realistic than random splitting.

2–3 Limitations

1. **Missing-data treatment:** forward fill can carry an old measurement through a long missing interval.
2. **Model limitation:** Random Forest does not explicitly model sequential dynamics as dedicated time-series/deep-learning models do.
3. **Validation limitation:** only one chronological holdout and one forecast horizon were evaluated.

Possible improvement for future work

Compare Random Forest with gradient boosting and dedicated time-series/deep-learning models, use rolling-origin cross-validation and systematic hyperparameter tuning, and evaluate multiple horizons such as 1-hour, 6-hour and 24-hour ahead. External variables such as traffic and weather forecasts could also be incorporated where available.

Conclusion

The pipeline provides a leakage-aware one-hour-ahead CO forecasting workflow using the supplied UCI Air Quality dataset. It combines preprocessing, temporal feature engineering, chronological validation, Random Forest regression, standard regression metrics, error analysis and feature-importance analysis.